

LeadOS Backend Implementation Plan

1. Purpose

This document explains exactly how the LeadOS backend should be built for the MVP.

The backend is the core of the product. It must connect all user facing apps, Supabase Auth, Supabase Postgres, AI chat, Vapi voice events, scoring, routing, RM handoff, WhatsApp link generation, notifications, analytics, and knowledge documents.

The MVP must be completed fast, so the backend should be clean, practical, and not over engineered.

The backend must support this complete flow:

Lead created → AI chat or voice conversation → transcript stored → scoring runs → lead classified Hot, Warm, or Cold → Hot lead assigned to RM → Warm lead gets WhatsApp follow up link → Cold lead is closed → Admin and RM portals update

2. Backend Tech Stack

Use:

Supabase Auth
Supabase Postgres
Supabase Edge Functions
TypeScript
Deno runtime for Edge Functions
OpenAI for chat, scoring, and structured extraction
Vapi for voice calls and webhooks
LangGraph style conversation state logic
LangChain style document retrieval if time allows
Mem0 optional, fallback to Supabase JSON memory
n8n optional for automation, fallback to backend service logic

Important MVP decision:

Frontend will never call Supabase tables directly.
Frontend will only call Edge Function APIs.
Edge Functions will validate auth, role, client scope, and then access Supabase Postgres server side.

3. Backend Responsibilities

The backend must handle:

Authentication verification
Role based access
Client scoped data access
Lead creation
Lead listing and detail
Chat conversation lifecycle
Voice webhook handling
Conversation transcript storage
AI response generation
Language handling
Knowledge context loading
Scoring
Hot, Warm, Cold routing
RM task creation
WhatsApp wa.me link creation
Follow up tracking
Notifications
Analytics APIs
Knowledge document APIs
RM feedback APIs

The backend should not handle:

Frontend UI state
Chart rendering
Portal layout
Payment or billing
Full CRM features
Full WhatsApp Business API
Pipecat implementation
Advanced ML training
Complex tenant onboarding

4. High Level Backend Architecture

Frontend Apps
→ Edge Function API Router
→ Auth Middleware
→ Role Guard
→ Controller Layer
→ Service Layer
→ Repository Layer
→ Supabase Postgres

AI Services
→ OpenAI
→ Vapi Webhooks
→ Knowledge Documents
→ Scoring Engine
→ Routing Engine

Main backend layers:

1. API Router
2. Middleware
3. Controllers
4. Services
5. Repositories
6. AI Modules
7. Utility Modules
8. Database Access

5. Supabase Edge Function Structure

For MVP, use two main Edge Functions.

```
/functions/v1/api  
/functions/v1/vapi-webhook
```

5.1 Main API Function

Path:

```
/functions/v1/api
```

Purpose:

All authenticated and public app APIs.

Examples:

```
POST /functions/v1/api/auth/login  
POST /functions/v1/api/public/leads  
GET /functions/v1/api/leads  
POST /functions/v1/api/chat/message  
GET /functions/v1/api/analytics/overview
```

5.2 Vapi Webhook Function

Path:

```
/functions/v1/vapi-webhook
```

Purpose:

Receive Vapi events.

Examples:

```
Transcript event  
Call ended event  
Tool call event if needed
```

This function is public but protected by webhook secret verification.

6. Recommended Folder Structure

```
supabase/  
  functions/  
    api/  
      index.ts  
      router.ts  
  
    middleware/  
      auth.ts  
      roleGuard.ts  
      publicSession.ts  
      errorHandler.ts  
      validateBody.ts  
      requestLogger.ts  
  
    controllers/  
      auth.controller.ts  
      public.controller.ts  
      lead.controller.ts  
      conversation.controller.ts  
      chat.controller.ts  
      voice.controller.ts  
      scoring.controller.ts  
      rmTask.controller.ts  
      followUp.controller.ts  
      notification.controller.ts  
      analytics.controller.ts  
      campaign.controller.ts  
      user.controller.ts  
      knowledge.controller.ts  
      feedback.controller.ts  
  
    services/  
      auth.service.ts  
      publicLead.service.ts  
      lead.service.ts  
      conversation.service.ts  
      chatAgent.service.ts  
      language.service.ts  
      knowledge.service.ts  
      rag.service.ts  
      memory.service.ts  
      scoring.service.ts  
      routing.service.ts  
      rmTask.service.ts  
      followUp.service.ts  
      notification.service.ts  
      analytics.service.ts  
      campaign.service.ts  
      user.service.ts  
      feedback.service.ts
```

```
repositories/  
  client.repository.ts  
  user.repository.ts  
  campaign.repository.ts  
  lead.repository.ts  
  conversation.repository.ts  
  message.repository.ts  
  score.repository.ts  
  rmTask.repository.ts  
  followUp.repository.ts  
  notification.repository.ts  
  knowledge.repository.ts  
  feedback.repository.ts  
  
ai/  
  prompts/  
    conversation.prompt.ts  
    scoring.prompt.ts  
  agent/  
    conversationState.ts  
    agentRunner.ts  
    tools.ts  
  scoring/  
    scoringRunner.ts  
    scoringValidator.ts  
  openai/  
    openaiClient.ts  
  
utils/  
  response.ts  
  errors.ts  
  pagination.ts  
  filters.ts  
  sorting.ts  
  validation.ts  
  waMe.ts  
  phone.ts  
  date.ts  
  env.ts  
  logger.ts  
  
types/  
  api.types.ts  
  auth.types.ts  
  lead.types.ts  
  conversation.types.ts  
  scoring.types.ts  
  routing.types.ts  
  
vapi-webhook/  
  index.ts  
  verifyWebhook.ts  
  handlers/  
    transcript.handler.ts  
    callEnded.handler.ts
```

If this feels too much for four days, use fewer files but keep the same logical separation.

7. Environment Variables

Required environment variables:

```
SUPABASE_URL
SUPABASE_SERVICE_ROLE_KEY
SUPABASE_ANON_KEY
OPENAI_API_KEY
VAPI_API_KEY
VAPI_WEBHOOK_SECRET
DEFAULT_CLIENT_SLUG
DEFAULT_CAMPAIGN_ID
DEFAULT_RM_ID
DEMO_ONBOARDING_LINK
```

Optional:

```
MEMO_API_KEY
N8N_WEBHOOK_URL
APP_BASE_URL
```

Important:

Never expose service role key to frontend.
Only Edge Functions use service role key.
Frontend uses only API endpoints.

8. Database Tables Used

The backend will use these MVP tables:

```
clients
users
campaigns
leads
conversations
conversation_messages
lead_scores
rm_tasks
follow_ups
notifications
knowledge_documents
rm_feedback
```

Optional:

```
knowledge_chunks
```

9. Authentication and Authorization

9.1 Auth Model

We use Supabase Auth for Admin and RM users.

Frontend login:

```
POST /auth/login
```

Backend:

```
Validate credentials through Supabase Auth
Fetch user profile from users table
Return token and profile
```

Every protected request sends:

```
Authorization: Bearer access_token
```

Edge Function must:

```
Read token
Verify token with Supabase Auth
Fetch users row
Attach auth context to request
```

9.2 Auth Context

Create an internal context object:

```
type AuthContext = {
  userId: string;
  clientId: string;
  role: "admin" | "rm";
  email: string;
  name: string;
};
```

This must be available to controllers and services.

9.3 Role Rules

Admin can:

```
Read all client leads
Read all client conversations
Read all client scores
```

Read all client analytics
Manage users
Manage campaigns
Manage knowledge documents
Read feedback

RM can:

Read assigned leads
Read own RM tasks
Read conversations for assigned leads
Read scores for assigned leads
Update own task status
Submit feedback
Read own notifications

Public lead can:

Create lead
Start chat for own lead
Send chat message for own lead
End own chat
Start voice request for own lead

Public access must use `leadSessionToken`.

10. API Response Format

Use one consistent response format.

10.1 Success

```
{  
  "success": true,  
  "data": {},  
  "meta": {}  
}
```

10.2 List

```
{  
  "success": true,  
  "data": [],  
  "pagination": {  
    "page": 1,  
    "limit": 20,  
    "total": 100,  
    "totalPages": 5  
  },  
  "sort": {  
    "sortBy": "created_at",  
    "sortOrder": "desc"  
  }  
}
```



```
    },  
    "filters": {}  
  }  
}
```

10.3 Error

```
{  
  "success": false,  
  "error": {  
    "code": "LEAD_NOT_FOUND",  
    "message": "Lead not found",  
    "details": {}  
  },  
  "requestId": "req_123"  
}
```

11. Error Codes

Use predictable error codes.

```
UNAUTHORIZED  
FORBIDDEN  
VALIDATION_ERROR  
NOT_FOUND  
CLIENT_NOT_FOUND  
USER_NOT_FOUND  
LEAD_NOT_FOUND  
CONVERSATION_NOT_FOUND  
TASK_NOT_FOUND  
FOLLOW_UP_NOT_FOUND  
INVALID_STATUS_TRANSITION  
INVALID_CLASSIFICATION  
SCORING_FAILED  
AI_RESPONSE_FAILED  
VAPI_WEBHOOK_INVALID  
RATE_LIMITED  
INTERNAL_ERROR
```

12. Middleware Plan

12.1 requestLogger

Purpose:

Log method, path, request id, duration, status

12.2 authMiddleware

Purpose:

Verify Supabase Auth token
Load user profile
Attach AuthContext

12.3 roleGuard

Purpose:

Allow only certain roles for endpoint

Example:

```
requireRole(["admin"]);  
requireRole(["admin", "rm"]);
```

12.4 publicLeadSessionMiddleware

Purpose:

Verify leadSessionToken for public chat and voice routes
Ensure token belongs to lead
Prevent public user from accessing other leads

12.5 validateBody

Purpose:

Validate request body using Zod
Return 422 if invalid

12.6 errorHandler

Purpose:

Catch errors
Return standard error response

13. Validation Strategy

Use Zod for all request validation.

Create schemas for:

Login request
Create public lead
Create lead
Lead update
Start chat
Send chat message

End chat
Start voice
Run scoring
Task update
Follow up actions
Feedback submit
Create RM
Create campaign
Create knowledge document

Example:

```
const CreatePublicLeadSchema = z.object({
  clientSlug: z.string().min(1),
  campaignId: z.string().uuid().optional(),
  name: z.string().min(2),
  phone: z.string().min(10),
  email: z.string().email().optional(),
  city: z.string().optional(),
  role: z.string().optional(),
  networkSize: z.string().optional(),
  preferredLanguage: z.enum(["english", "hindi", "hinglish"]),
  preferredChannel: z.enum(["chat", "voice"]),
  whatsappConsent: z.boolean().default(false),
  source: z.string().default("website")
});
```

14. Repository Layer

Repositories should only handle database operations.

They should not contain business logic.

14.1 LeadRepository

Methods:

```
createLead(input)
findById(leadId, clientId)
listLeads(params)
updateLead(leadId, clientId, patch)
assignRM(leadId, clientId, rmId)
countByStatus(clientId)
countByClassification(clientId)
```

14.2 ConversationRepository

Methods:

```
createConversation(input)
findById(conversationId, clientId)
```

```
findLatestByLeadId(leadId, clientId)
updateConversation(conversationId, clientId, patch)
completeConversation(conversationId, clientId, patch)
```

14.3 MessageRepository

Methods:

```
createMessage(input)
listByConversation(conversationId, clientId)
getNextSequenceNo(conversationId)
compileTranscript(conversationId, clientId)
```

14.4 ScoreRepository

Methods:

```
createScore(input)
findLatestByLeadId(leadId, clientId)
listByLeadId(leadId, clientId)
countByClassification(clientId)
```

14.5 RMTaskRepository

Methods:

```
createTask(input)
listTasks(params)
findById(taskId, clientId)
updateTask(taskId, clientId, patch)
findByLeadId(leadId, clientId)
```

14.6 FollowUpRepository

Methods:

```
createFollowUp(input)
listFollowUps(params)
findById(followUpId, clientId)
updateFollowUp(followUpId, clientId, patch)
findByLeadId(leadId, clientId)
```

14.7 NotificationRepository

Methods:

```
createNotification(input)
listForUser(userId, clientId, params)
markRead(notificationId, userId)
markAllRead(userId, clientId)
```

14.8 KnowledgeRepository

Methods:

```
listActiveDocuments(clientId)
listDocuments(clientId, params)
findById(documentId, clientId)
createDocument(input)
```

14.9 FeedbackRepository

Methods:

```
createFeedback(input)
listFeedback(clientId, params)
listByRM(rmId, clientId, params)
```

15. Service Layer

Services contain business logic.

15.1 AuthService

Responsibilities:

```
Login user
Refresh session
Get current user
Validate active user
Return user and client profile
```

Methods:

```
login(email, password)
refresh(refreshToken)
getCurrentUser(accessToken)
logout(refreshToken)
```

15.2 PublicLeadService

Responsibilities:

```
Create lead from dummy client website
Resolve client by slug
Attach default campaign
Create lead session token
Return next action
```

Flow:

```
Validate clientSlug  
Find client  
Find campaign  
Create lead  
Set status = pending_contact  
Generate leadSessionToken  
Return lead and token
```

15.3 LeadService

Responsibilities:

```
Create lead  
List leads  
Get lead detail  
Update lead  
Assign RM  
Enforce role based access
```

Important access logic:

```
Admin sees all client leads.  
RM sees only leads assigned to them or linked to their RM tasks.
```

Lead detail must assemble:

```
lead  
campaign  
assigned RM  
latest conversation  
messages  
latest score  
RM task  
follow up  
feedback optional
```

15.4 ConversationService

Responsibilities:

```
Create conversation  
Append messages  
Compile transcript  
Finalize conversation  
Fetch latest conversation
```

When conversation completes:

```
Set conversation.status = completed
Set ended_at
Set transcript_text
Set summary if available
Set lead.status = conversation_completed
Trigger scoring
```

15.5 ChatAgentService

Responsibilities:

```
Start chat
Send message
Generate assistant response
Store messages
End chat
Trigger scoring
```

Uses:

```
ConversationService
AgentOrchestratorService
KnowledgeService
MemoryService
OpenAIClient
```

15.6 AgentOrchestratorService

Responsibilities:

```
Load lead context
Load conversation history
Load memory
Load knowledge context
Build prompt
Call OpenAI
Parse tool requests if any
Return assistant response
```

MVP conversation stages:

```
Greeting
Language confirmation
Hook
Pitch
Discovery
Objection handling
Qualification
Close
```

For MVP, stage can be stored in:

`conversations.metadata.current_stage`

15.7 LanguageService

Responsibilities:

Use selected language
Detect switch if user asks
Update language when needed

Methods:

`resolveInitialLanguage(lead)`
`detectLanguageSwitch(message)`
`normalizeLanguage(language)`

15.8 KnowledgeService

Responsibilities:

Load active knowledge documents
Return script, FAQ, onboarding, objection guide

MVP approach:

Fetch all active docs for client
Inject into prompt as compact context

If real RAG is not ready, use full seeded docs.

15.9 RAGService

Optional for MVP.

Responsibilities:

Find relevant chunks
Return top context

Fast MVP fallback:

Simple keyword search over `knowledge_documents.content`

15.10 MemoryService

Responsibilities:

Get lead memory
Save language preference
Save callback time
Save WhatsApp consent
Save latest summary

MVP storage:

leads.metadata

Optional:

Mem0Client

15.11 ScoringService

Responsibilities:

Build scoring input
Call scoring LLM
Validate JSON output
Insert lead_scores
Update lead_latest fields
Call RoutingService

Methods:

runScoring(leadId, conversationId, clientId)
buildScoringInput(lead, conversation, messages)
callScoringLLM(input)
validateScoringOutput(output)
persistScore(output)

15.12 RoutingService

Responsibilities:

Route Hot to RM task
Route Warm to follow up
Route Cold to closed

Methods:

routeLead(score)
routeHotLead(lead, score)
routeWarmLead(lead, score)
routeColdLead(lead, score)

15.13 RMTaskService

Responsibilities:

- Create RM task
- List RM tasks
- Update RM task status
- Update lead outcome when task converted or closed

Hot lead task should include:

- recommended_action
- suggested_opening_line
- priority = high
- status = pending

15.14 FollowUpService

Responsibilities:

- Generate WhatsApp message
- Generate wa.me link
- Create follow up
- Open follow up
- Mark sent
- Cancel follow up

15.15 NotificationService

Responsibilities:

- Create RM notification
- List notifications
- Mark read
- Mark all read

Notification examples:

- New Hot Lead Assigned
- Follow Up Ready
- Follow Up Due
- Score Ready

15.16 AnalyticsService

Responsibilities:

- Overview metrics
- Funnel
- Language stats
- Objection stats

RM performance
Campaign stats

All analytics should be calculated from existing tables.

No metrics tables for MVP.

15.17 UserService

Responsibilities:

List users
List RMs
Create RM
Deactivate user
Reactivate user optional

For MVP, Supabase Auth user creation can be manual if needed.

If time is tight:

Seed RMs manually
Only implement list users and deactivate mock ready endpoint

15.18 FeedbackService

Responsibilities:

Submit RM feedback
List feedback for admin

Feedback does not auto update scoring in MVP.

16. API Implementation Plan

16.1 Auth APIs

POST /auth/login

Controller:

AuthController.login

Service:

AuthService.login

Tables:

auth.users
users
clients

Steps:

Validate email and password
Call Supabase Auth signInWithPassword
Fetch users profile
Fetch client
Return token and user data

GET /auth/me

Steps:

Verify token
Fetch user profile
Fetch client
Return context

POST /auth/logout

Steps:

Call Supabase Auth signOut if needed
Return success

16.2 Public APIs

GET /public/client/:clientSlug

Steps:

Find client by slug
Find default campaign
Return supported languages and channels

POST /public/leads

Steps:

Validate body
Find client
Find campaign
Create lead with status pending_contact

Generate leadSessionToken
Return lead and nextAction

Lead session token options:

Option 1: Signed JWT using Edge Function secret
Option 2: Random token stored in leads.metadata.public_session_token

For MVP, use Option 2.

Store:

```
{  
  "public_session_token": "random_token"  
}
```

16.3 Lead APIs

GET /leads

Steps:

Auth required
Apply role scope
Apply filters
Apply sorting
Apply pagination
Return list

Filters:

search
status
classification
language
campaignId
assignedRmId
source
fromDate
toDate

Sorting:

created_at
updated_at
latest_score
last_contacted_at
name
status
classification

GET /leads/:leadId

Steps:

- Auth required
- Check access
- Fetch lead
- Fetch campaign
- Fetch assigned RM
- Fetch latest conversation
- Fetch messages
- Fetch latest score
- Fetch RM task
- Fetch follow up
- Return assembled response

PATCH /leads/:leadId

Steps:

- Auth required
- Check role
- Validate allowed fields
- Update lead
- Return updated lead

POST /leads/:leadId/assign-rm

Steps:

- Admin only
- Validate RM belongs to same client
- Update lead.assigned_rm_id
- Optionally create notification
- Return success

16.4 Chat APIs

POST /chat/start

Steps:

- Validate public lead session
- Fetch lead
- Create conversation with channel chat
- Set language from lead.preferred_language
- Create assistant greeting message
- Store assistant message
- Return conversation and assistant message

POST /chat/message

Steps:

```
Validate public lead session
Fetch conversation
Store user message
Load conversation history
Load lead context
Load knowledge docs
Run agent
Store assistant response
Return assistant response and current stage
```

POST /chat/end

Steps:

```
Validate public lead session
Fetch conversation messages
Compile transcript
Set conversation completed
Set lead status conversation_completed
Run scoring
Run routing
Return score and routing result
```

16.5 Voice APIs

POST /voice/start

Steps:

```
Validate public lead session
Create conversation with channel voice
Call Vapi to create session or call
Store provider_conversation_id
Return join URL or session info
```

If Vapi is not ready:

```
Return simulated voice request accepted
Keep status pending_contact or started based on UI
```

POST /vapi-webhook

Steps:

```
Verify webhook secret
Parse event type
```

If transcript event, store message
If call ended event, complete conversation
Run scoring
Run routing
Return received

Important:

Vapi webhook must never trust client supplied IDs blindly.
Use `provider_conversation_id` to map call to conversation.

16.6 Scoring APIs

POST /scores/run

Steps:

Auth or internal call
Fetch lead
Fetch conversation
Fetch messages
Build scoring input
Call scoring LLM
Validate output
Insert lead_scores
Update lead latest fields
Run routing
Return score and routing

This endpoint is useful for manual testing.

16.7 RM Task APIs

GET /rm/tasks

Steps:

Auth required
Admin sees all tasks
RM sees own tasks
Apply filters and pagination
Return tasks with lead summary

GET /rm/tasks/:taskId

Steps:

Fetch task
Check access

Fetch lead, latest score, conversation, messages
Return detail

PATCH /rm/tasks/:taskId

Steps:

Check access
Update task status
If converted, update lead.status converted and converted_at
If closed, update lead.status closed and closed_at
Return task and lead

16.8 Follow Up APIs

GET /followups

Steps:

Auth required
Admin sees all client follow ups
RM sees follow ups for assigned leads or own follow ups
Apply filters
Return list

POST /followups/:id/open

Steps:

Check access
Set status opened
Set opened_at
Return wa_me_link

POST /followups/:id/mark-sent

Steps:

Check access
Set status sent
Set sent_at
Return success

16.9 Notification APIs

GET /notifications

Steps:

Fetch notifications for auth user
Apply isRead filter
Return list

PATCH /notifications/:id/read

Steps:

Check owner
Set is_read true
Set read_at

PATCH /notifications/read-all

Steps:

Mark all notifications for user as read

16.10 Analytics APIs

GET /analytics/overview

Return:

totalLeads
conversationCompleted
hot
warm
cold
assignedToRm
followUpsScheduled
converted
closed

GET /analytics/funnel

Return stages:

new
pending_contact
conversation_completed
assigned_to_rm
follow_up_scheduled
converted
closed

GET /analytics/languages

Group by:

`leads.preferred_language`

GET /analytics/objections

Read:

`lead_scores.objections jsonb`

Count objection types and resolution statuses.

GET /analytics/rm-performance

Group by RM:

`assigned leads
converted
closed
pending tasks`

17. AI Conversation Implementation

17.1 Chat Start Prompt

On chat start, build a greeting using:

`lead name
preferred language
role if known
program context`

Example:

Hi Amit, thanks for your interest. I can quickly explain the partner program in Hinglish and see if it fits you.

17.2 Chat Message Prompt Context

Every AI call should include:

`System prompt
Lead profile
Conversation stage
Conversation history
Memory
Knowledge context
Latest user message
Available tools`

17.3 Conversation Stage Storage

Store current stage in:

```
conversations.metadata.current_stage
```

Allowed stages:

```
greeting  
language_confirmation  
hook  
pitch  
discovery  
objection_handling  
qualification  
close  
ended
```

17.4 Stage Update

Agent should return:

```
{  
  "assistant_message": "text",  
  "stage": "objection_handling",  
  "language": "hinglish",  
  "tool_calls": []  
}
```

Backend updates:

```
conversation.metadata.current_stage  
conversation.language if changed
```

17.5 Tool Calls During Conversation

MVP live tools:

```
update_language  
save_whatsapp_consent  
save_callback_time
```

Post scoring tools:

```
create_rm_task  
generate_whatsapp_followup  
assign_rm
```

For MVP, tool calls can be implemented as simple backend detection from structured output instead of full OpenAI tool calling if time is short.

18. Scoring Implementation

18.1 Scoring Input Builder

Collect:

```
Lead profile
Conversation channel
Conversation language
Duration
Transcript messages
Lead metadata
Previous summary if any
```

Build compact JSON input.

18.2 OpenAI Structured Output

Use OpenAI JSON mode or strict structured output.

Expected schema:

```
type ScoringOutput = {
  classification: "hot" | "warm" | "cold";
  total_score: number;
  readiness_score: number;
  engagement_score: number;
  fit_score: number;
  reason: string;
  recommended_next_action: string;
  suggested_opening_line: string | null;
  positive_signals: string[];
  negative_signals: string[];
  objections: {
    type:
      | "existing_broker"
      | "not_enough_contacts"
      | "client_support_concern"
      | "trust_concern"
      | "call_later"
      | "other"
      | "none";
    lead_statement: string;
    status: "resolved" | "partially_resolved" | "unresolved";
    ai_response_summary: string;
    impact_on_score: string;
  }[];
```

```
    extracted_data: Record<string, unknown>;  
};
```

18.3 Backend Validation

Check:

```
classification is valid  
scores are in ranges  
total score equals sum  
reason exists  
recommended_next_action exists  
arrays are arrays
```

If JSON fails:

```
Retry once with repair prompt  
If still fails, fallback to Warm manual review style score
```

Fallback:

```
{  
  "classification": "warm",  
  "total_score": 50,  
  "readiness_score": 18,  
  "engagement_score": 18,  
  "fit_score": 14,  
  "reason": "Scoring could not be completed reliably. Manual review  
recommended.",  
  "recommended_next_action": "RM should review transcript.",  
  "suggested_opening_line": null,  
  "positive_signals": [],  
  "negative_signals": ["Scoring fallback used"],  
  "objections": [],  
  "extracted_data": {}  
}
```

19. Routing Implementation

Routing runs after scoring.

19.1 Hot Routing

Steps:

```
Find or assign RM  
Update lead classification hot  
Update lead status assigned_to_rm  
Create RM task
```

```
Create RM notification
Return routing type rm_handoff
```

RM assignment MVP strategy:

```
If lead.assigned_rm_id exists, use it.
Else assign DEFAULT_RM_ID.
```

19.2 Warm Routing

Steps:

```
Update lead classification warm
Update lead status follow_up_scheduled
Generate WhatsApp message
Generate wa.me link
Create follow_ups row
Return routing type whatsapp_followup
```

19.3 Cold Routing

Steps:

```
Update lead classification cold
Update lead status closed
Set closed_at
Return routing type closed
```

20. WhatsApp Link Generation

Use wa.me format.

Function:

```
function generateWaMeLink(phone: string, message: string): string {
  const normalizedPhone = normalizeIndianPhone(phone);
  const encoded = encodeURIComponent(message);
  return `https://wa.me/${normalizedPhone}?text=${encoded}`;
}
```

Phone normalization:

```
Remove spaces
Remove plus
If 10 digits, prefix 91
If already starts with 91, keep
```

Message template:

Hi {{name}}, thanks for speaking with our AI assistant about the partner program. Here are the details you asked for: {{demoLink}}. You can reply here if you would like a relationship manager to guide you.

21. n8n Usage Plan

n8n is optional in backend MVP.

21.1 Ideal n8n workflows

Lead created workflow
Hot lead notification workflow
Warm follow up workflow
Callback reminder workflow

21.2 Practical MVP decision

If n8n setup takes time:

Implement routing directly in backend.
Keep n8n as future workflow layer.

Do not let n8n block demo.

22. RLS Implementation Plan

Even though frontend does not directly call Supabase DB, enable RLS for safety.

22.1 General policy

No anon direct access to tables.
Authenticated users only see rows in their client scope.
Admins see all client rows.
RMs see only assigned lead related rows.
Service role in Edge Functions can bypass RLS, but API must enforce access manually.

22.2 MVP practical approach

Enable RLS on all tables.
Create restrictive default policies.
Use service role in Edge Functions.
Implement role and scope checks in backend.

This is fast and safe enough for MVP.

23. List Filtering and Sorting Implementation

Create helper:

```
type ListParams = {
  page?: number;
  limit?: number;
  search?: string;
  sortBy?: string;
  sortOrder?: "asc" | "desc";
  filters?: Record<string, string | undefined>;
};
```

Pagination:

```
page default 1
limit default 20
max limit 100
offset = (page minus 1) times limit
```

Sorting:

```
sortBy must be whitelisted per endpoint
sortOrder must be asc or desc
```

Never allow arbitrary column names without whitelist.

24. Request and Response Helpers

Create:

```
success(data, meta?)
listSuccess(data, pagination, sort, filters)
error(code, message, status, details?)
```

Example:

```
return json(success({ lead }), 200);
```

25. Implementation Day Plan

This is the practical four day backend plan.

Day 1: Foundation and Core CRUD

Goal:

Auth, DB access, public lead creation, lead list, lead detail

Tasks:

Set up Supabase project
Create database tables
Seed client, campaign, admin, RM, knowledge docs
Create api Edge Function
Create router
Create response helpers
Create error helpers
Create auth middleware
Create role guard
Create Supabase server client helper
Implement auth login
Implement auth me
Implement public client config
Implement public lead creation
Implement lead list
Implement lead detail
Implement basic users list
Test with Postman or Bruno

Deliverable by end of Day 1:

Admin and RM can log in
Dummy website can create lead
Admin can list leads
Admin and RM can open lead detail

Day 2: Chat, Conversation, Knowledge

Goal:

Working AI chat pipeline with transcript storage

Tasks:

Create conversation service
Create message repository
Implement chat start
Implement chat message
Implement chat end
Create base conversation system prompt
Load lead profile
Load knowledge documents
Store user messages
Store assistant messages
Compile transcript
Update conversation status
Update lead status conversation_completed
Build simple language handling
Build simple stage tracking in conversation metadata
Test complete chat flow

Deliverable by end of Day 2:

Lead can chat with AI
Messages are stored
Transcript is saved
Conversation can end
Lead status updates

Day 3: Scoring, Routing, Follow Up, RM Task

Goal:

Conversation ends and automatically becomes Hot, Warm, or Cold with routing

Tasks:

Create scoring prompt
Implement scoring input builder
Implement OpenAI scoring call
Implement scoring output validation
Insert lead_scores
Update leads latest_score, classification, summary, next_action
Implement routing service
Hot creates RM task
Hot creates notification
Warm creates follow up with wa.me link
Cold closes lead
Implement manual POST /scores/run
Implement RM tasks list
Implement followups list
Implement notifications list
Test Hot scenario
Test Warm scenario
Test Cold scenario

Deliverable by end of Day 3:

End chat → scoring runs → routing happens → Admin and RM data available

Day 4: Voice Webhook, Analytics, Polish, Stability

Goal:

Demo ready backend

Tasks:

Implement Vapi webhook function
Store transcript events
Handle call ended event
Trigger scoring after call end
Implement analytics overview
Implement funnel stats
Implement language stats
Implement objection stats
Implement RM performance stats
Implement feedback submit
Implement follow up open and mark sent
Implement task update converted or closed
Add error handling
Add logs
Add fallback if OpenAI fails
Add seeded demo data
Run full demo test

Deliverable by end of Day 4:

Backend supports full demo path
Chat path stable
Voice webhook path available
Admin and RM portals can integrate

26. Endpoint Build Priority

Build in this exact order:

1. POST /auth/login
2. GET /auth/me
3. GET /public/client/:clientSlug
4. POST /public/leads
5. GET /leads
6. GET /leads/:leadId
7. POST /chat/start
8. POST /chat/message
9. POST /chat/end
10. POST /scores/run
11. GET /rm/tasks
12. PATCH /rm/tasks/:taskId
13. GET /followups
14. POST /followups/:id/open
15. POST /followups/:id/mark-sent
16. GET /notifications
17. PATCH /notifications/:id/read
18. GET /analytics/overview
19. GET /analytics/funnel
20. GET /analytics/languages
21. GET /analytics/objections
22. POST /vapi-webhook
23. GET /users
24. POST /users/rms
25. PATCH /users/:userId

26. GET /knowledge
27. GET /feedback
28. POST /feedback

27. Seed Data Plan

Seed one client:

FinPartner Pro

Seed one campaign:

Demo Partner Campaign

Seed users:

Sharad Admin
Shivansh RM
Second RM
Inactive RM

Seed knowledge docs:

Demo Sales Script
Demo FAQ
Demo Onboarding Guide
Demo Objection Guide

Seed example leads:

Amit Sharma → Hot
Priya Mehta → Warm
Rahul Verma → Cold
Neha Rao → Pending Contact
Kabir Khan → Conversation Completed

Seeded data helps frontends develop even before live AI is ready.

28. Testing Checklist

28.1 Auth Tests

Admin login works
RM login works
Invalid login fails
Inactive user blocked
Auth me returns correct profile
RM cannot access admin only endpoint

28.2 Lead Tests

Public lead creation works
Lead status becomes pending_contact
Admin can list all leads
RM only sees assigned leads
Lead detail includes score, messages, task, follow up
Filters work
Sorting works
Pagination works

28.3 Chat Tests

Start chat creates conversation
Assistant greeting stored
User message stored
Assistant response stored
End chat completes conversation
Transcript compiled
Lead status becomes conversation_completed

28.4 Scoring Tests

Hot transcript scores Hot
Warm transcript scores Warm
Cold transcript scores Cold
Score JSON validates
Invalid JSON retries
Fallback works
lead_scores row created
leads latest fields updated

28.5 Routing Tests

Hot creates RM task
Hot creates notification
Warm creates follow up
Warm creates wa.me link
Cold sets lead closed
Converted task updates lead converted
Closed task updates lead closed

28.6 Analytics Tests

Overview counts correct
Funnel counts correct
Language stats correct
Objection stats read JSON correctly
RM performance correct

28.7 Voice Tests

Webhook secret verified
Transcript event stored
Call ended event completes conversation
Scoring triggers after call end
Invalid webhook rejected

29. Demo Readiness Checklist

Before demo, verify:

Lead form creates lead
Chat starts
AI handles existing broker objection
Chat ends
Scoring runs
Lead becomes Hot
RM task appears
Notification appears
Admin dashboard updates
RM lead detail shows score, transcript, objections, suggested opening line
Warm lead creates WhatsApp link
Follow up page opens wa.me link
Voice webhook can be demonstrated or explained

30. Fallback Plan

30.1 If OpenAI chat fails

Use fallback response:

I understand. A relationship manager can explain this better and help with the next step.

Still store message.

30.2 If scoring fails

Use fallback score:

Warm
50
Manual review recommended

30.3 If Vapi fails

Use chat demo as primary.

Say:

The same conversation pipeline powers voice through Vapi. Chat is shown as the reliable demo path.

30.4 If n8n fails

Use backend routing directly.

30.5 If WhatsApp integration is questioned

Show wa.me link generation.

Say:

For MVP we generate the WhatsApp deep link automatically. In production this workflow can connect to Meta Cloud API, Twilio, or WATI.

31. Security Notes

Implement:

- No service role key in frontend
- All DB access through Edge Functions
- Auth required for Admin and RM
- Public lead session token for lead chat
- Webhook secret for Vapi
- Role checks on every protected endpoint
- Client scope checks on every protected endpoint
- Input validation on every endpoint
- No unrestricted SQL from frontend
- No direct Supabase table calls from frontend

Do not store:

- OTP
- Passwords
- Sensitive financial data
- Raw unnecessary personal data beyond MVP fields

32. Performance Notes

For MVP:

- Keep list APIs paginated
- Limit chat history passed to AI if too long

Use compact knowledge context
Use latest conversation for lead detail
Avoid expensive analytics until needed
Cache static knowledge docs in memory inside Edge Function if possible

33. Logging Notes

Log:

Request id
Endpoint
User id if available
Lead id if available
Conversation id if available
Scoring start and end
Routing result
Webhook received
Errors

Do not log:

Full access tokens
Full service role key
Sensitive secrets

34. Backend Completion Criteria

Backend is complete for MVP if:

Admin and RM auth works
Public lead creation works
Lead list and detail work
Chat start, message, and end work
Conversation messages are stored
Transcript is compiled
Scoring runs and stores result
Hot creates RM task and notification
Warm creates WhatsApp follow up link
Cold closes lead
RM can update task outcome
Admin analytics APIs work
Vapi webhook can store transcript and trigger scoring
Feedback API works
All responses follow standard response format
All protected APIs enforce role and client scope

35. Final Backend Principle

Build the backend around one winning flow:

Lead → AI conversation → transcript → scoring → routing → RM handoff or
WhatsApp follow up → dashboard

Anything that does not support this flow is not MVP.